File Systems: FFS and LFS

CS 202: Advanced Operating Systems

What's wrong with the original Unix FS?

- Original UNIX FS was simple and elegant, but **slow**

- Problems:
  - i-nodes far from data
    - All i-nodes at the beginning of the disk, all data after that
  - i-nodes of directory not close together
  - Consecutive blocks of files not close together
  - Blocks too small
    - 512 bytes (matched sector size)
  - Limited: file name length, maximum file size, directory size, metadata, support for file system backups



Seagate Enterprise HDD

- SATA II (300MB/s in theory), 7200 RPM., seek time around 8 ms. What's the **latency** and **throughput** of accessing a 512B sector?

Seagate Enterprise HDD

- SATA II (300MB/s in theory), 7200 RPM., seek time around 8 ms. What's the **latency** and **throughput** of accessing a 512B sector?

Disk I/O time

$$T_{I/O} = T_{seek} + T_{rotation} + T_{transfer}$$

Seagate Enterprise HDD

- SATA II (300MB/s in theory), 7200 RPM., seek time around 8 ms. What's the **latency** and **throughput** of accessing a 512B sector?

Disk I/O time

$$T_{I/O} = T_{seek} + T_{rotation} + T_{transfer}$$

Latency: $8ms + \frac{1}{\frac{7200}{60}} \cdot \frac{1}{2} + \frac{512B}{300MB/sec}$

Seagate Enterprise HDD

- SATA II (300MB/s in theory), 7200 RPM., seek time around 8 ms. What's the **latency** and **throughput** of accessing a 512B sector?

Disk I/O time

$$T_{I/O} = T_{seek} + T_{rotation} + T_{transfer}$$

Latency: $8ms + \frac{1}{\frac{7200}{60}} \cdot \frac{1}{2} + \frac{512B}{300MB/sec}$

$$= 8ms + 4.17ms + 0.00167\text{ms} = 12.17\text{ms}$$

Seagate Enterprise HDD

- SATA II (300MB/s in theory), 7200 RPM., seek time around 8 ms. What's the **latency** and **throughput** of accessing a 512B sector?

Disk I/O time

$T_{I/O} = T_{seek} + T_{rotation} + T_{transfer}$

Latency: $8ms + \frac{1}{\frac{7200}{60}} \cdot \frac{1}{2} + \frac{512B}{300MB/sec}$

$= 8ms + 4.17ms + 0.00167\text{ms} = 12.17\text{ms}$

Throughput: transmitted data over time

Seagate Enterprise HDD

- SATA II (300MB/s in theory), 7200 RPM., seek time around 8 ms. What's the **latency** and **throughput** of accessing a 512B sector?

Disk I/O time

$T_{I/O} = T_{seek} + T_{rotation} + T_{transfer}$

Latency: $8ms + \frac{1}{\frac{7200}{60}} \cdot \frac{1}{2} + \frac{512B}{300MB/sec}$

$= 8ms + 4.17ms + 0.00167\text{ms} = 12.17\text{ms}$

Throughput: transmitted data over time

$= 0.5KB/0.01217s = 41.08\text{KB/s}$

Seagate Enterprise HDD

- SATA II (300MB/s in theory), 7200 RPM., seek time around 8 ms. What's the **latency** and **throughput** of accessing a 512B sector?

Disk I/O time

$T_{I/O} = T_{seek} + T_{rotation} + T_{transfer}$

Latency: $8ms + \frac{1}{\frac{7200}{60}} \cdot \frac{1}{2} + \frac{512B}{300MB/sec}$

$= 8ms + 4.17ms + 0.00167\text{ms} = 12.17\text{ms}$

Throughput: transmitted data over time

$= 0.5KB/0.01217s = 41.08\text{KB/s}$

Recall how many accesses required to read /home/foo/bar.c…

UFS, FFS, and LFS

- UFS is the traditional file system used in Unix and its derivatives
  - Uses the traditional block-based allocation method for storing files and directories on a disk
- FFS is an improved version of UFS, one of the first disk-aware FS exploring locality
  - Key technique: cylinder groups
  - Many optimizations: e.g., more sophisticated block allocation algorithm to minimize fragmentation and improve performance
- LFS is a newer file system that focuses on improving writes performance
  - Uses a log-based architecture instead of cylinder groups

Fast File System (FFS)

A Fast File System for UNIX*

Marshall Kirk McKusick, William N. Joy†,
Samuel J. Leffler‡, Robert S. Fabry

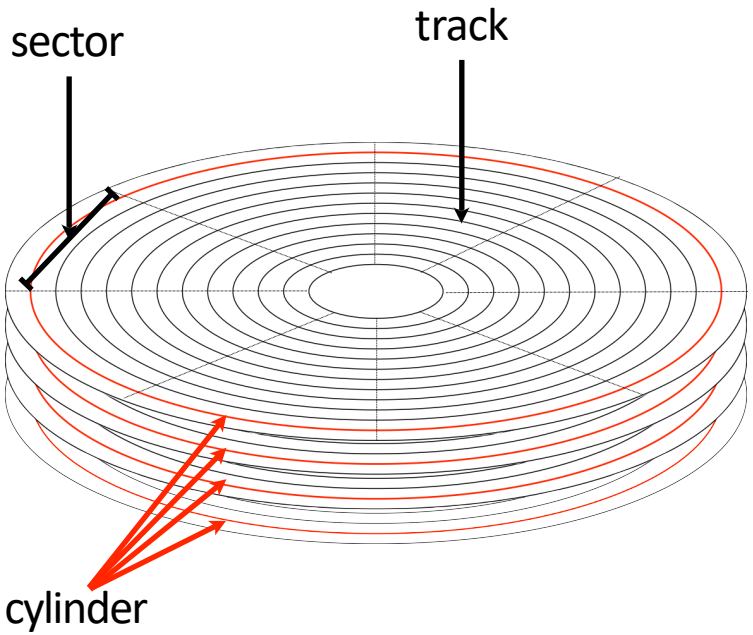
Computer Systems Research Group
Computer Science Division
Department of Electrical Engineering and Computer Science
University of California, Berkeley
Berkeley, CA 94720

Fast File System (FFS)

- FFS developed at Berkeley in 1984
  - First attempt at a disk aware file system
  - i.e. optimized for performance on spinning disks
- Observation: processes tend to access files that are in the same (or close) directories
  - Spatial locality
- Key idea: place groups of directories and their files into cylinder groups
  - Introduced into ext2, called *block groups*

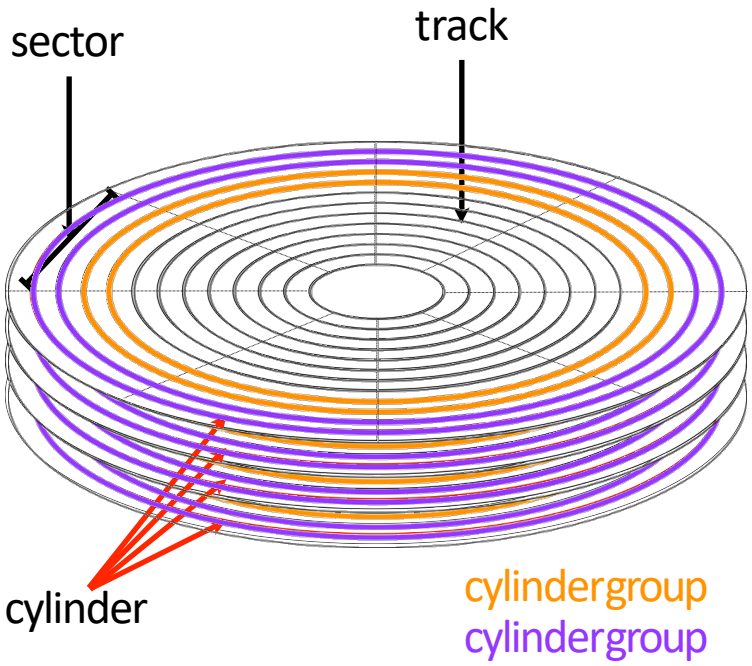
FFS Changes - Locality is important

- FFS Characteristics:
- Disk is divided into cylinder groups
  - Cylinder: a set of tracks on different disks with the same distance to the disk center
  - Cylinder group: *N* consecutive cylinders
- Inodes are spread across the disk
  - Keep inode near file
  - inodes of a directory together
- Larger block size
  - 4096 or 8192 bytes/block (why not larger?)



FFS Changes - Locality is important

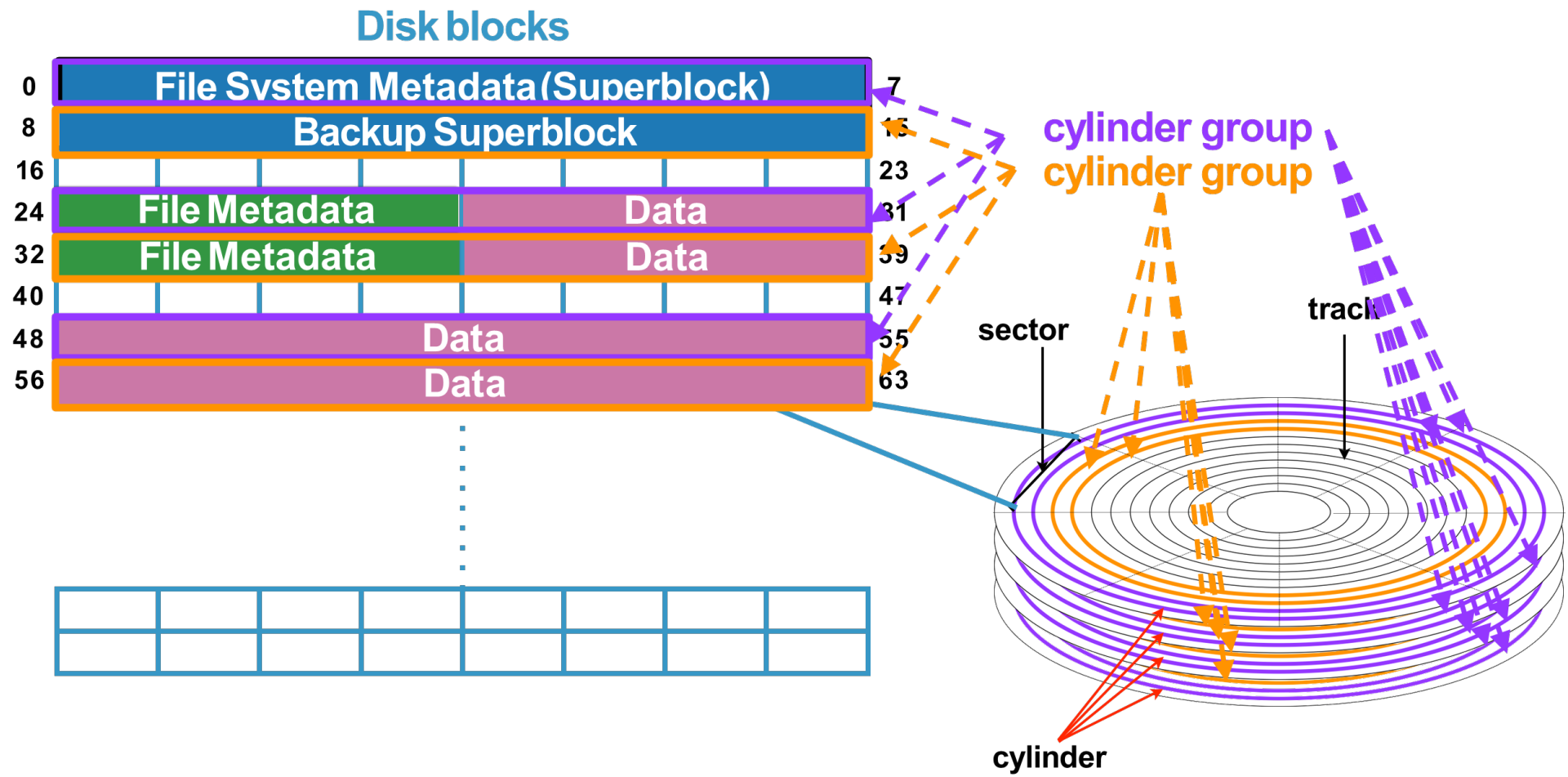
- FFS Characteristics:
- Disk is divided into cylinder groups
  - Cylinder: a set of tracks on different disks with the same distance to the disk center
  - Cylinder group: *N* consecutive cylinders
- Inodes are spread across the disk
  - Keep inode near file
  - inodes of a directory together
- Larger block size
  - 4096 or 8192 bytes/block (why not larger?)



Cylinder groups

- Consists of one or more consecutive cylinders on a disk
- Each cylinder group contains:
  - Redundant copy of the superblock (benefit?)
  - inode space
  - bitmap of free blocks within the cylinder group
  - summary of block usage
  - data blocks

How FFS use disk blocks



FFS Locality Techniques

- Goals
  - Allocate blocks of the same file within the same cylinder group
  - Place inode along with data within the same cylinder group
  - Keep directory within a cylinder group
    - Spread out different directories

- Layout policy: global and local
  - Global policy allocates files & directories to cylinder groups – picks "optimal" next block for block allocation
  - Local allocation routines handle specific block requests – select from a sequence of alternative if need to

Larger block sizes

- The block size of UFS is aligned with the sector size of the disk
  - Each file can only contain a fixed number of blocks
  - Cannot fully utilize the I/O interface bandwidth
- The new file system supports larger block sizes
  - Supports larger files
  - Each I/O request can carry more data to improve bandwidth
- However, larger block size leads to internal fragmentation

How larger block size improves bandwidth?

- SATA II (300MB/s in theory), 7200 RPM., seek time around 8 ms. What's the **latency** and **throughput** of accessing 4MB of consecutive sectors?

How larger block size improves bandwidth?

- SATA II (300MB/s in theory), 7200 RPM., seek time around 8 ms. What's the **latency** and **throughput** of accessing 4MB of consecutive sectors?

Latency: $8ms + \frac{1}{\frac{7200}{60}} \cdot \frac{1}{2} + \frac{4MB}{300MB/sec}$

How larger block size improves bandwidth?

- SATA II (300MB/s in theory), 7200 RPM., seek time around 8 ms. What's the **latency** and **throughput** of accessing 4MB of consecutive sectors?

Latency: $8ms + \frac{1}{\frac{7200}{60}} \cdot \frac{1}{2} + \frac{4MB}{300MB/sec}$

$= 8ms + 4.17ms + 13.33\text{ms} = 25.5\text{ms}$

How larger block size improves bandwidth?

- SATA II (300MB/s in theory), 7200 RPM., seek time around 8 ms. What's the **latency** and **throughput** of accessing 4MB of consecutive sectors?

Latency: $8ms + \frac{1}{\frac{7200}{60}} \cdot \frac{1}{2} + \frac{4MB}{300MB/sec}$

$= 8ms + 4.17ms + 13.33\text{ms} = 25.5\text{ms}$

Throughput: transmitted data over time

How larger block size improves bandwidth?

- SATA II (300MB/s in theory), 7200 RPM., seek time around 8 ms. What's the **latency** and **throughput** of accessing 4MB of consecutive sectors?

Latency: $8ms + \frac{1}{\frac{7200}{60}} \cdot \frac{1}{2} + \frac{4MB}{300MB/sec}$

$= 8ms + 4.17ms + 13.33\text{ms} = 25.5\text{ms}$

Throughput: transmitted data over time

$= 4MB/0.0255s = 155.86\text{MB/s}$

How larger block size improves bandwidth?

- SATA II (300MB/s in theory), 7200 RPM., seek time around 8 ms. What's the **latency** and **throughput** of accessing 4MB of consecutive sectors?

Latency: $8ms + \frac{1}{\frac{7200}{60}} \cdot \frac{1}{2} + \frac{4MB}{300MB/sec}$

$= 8ms + 4.17ms + 13.33\text{ms} = 25.5\text{ms}$

Throughput: transmitted data over time

$= 4MB/0.0255s = 155.86\text{MB/s}$

Trading latency with throughput

How to address internal fragmentation?

- Sub-blocks within a block
  - 512-byte little blocks

How to address internal fragmentation?

- Sub-blocks within a block
  - 512-byte little blocks

- How does it work?
  - If you created a small file (say 1KB in size), it would occupy two sub-blocks and thus not waste an entire 4KB block
  - As the file grew, the file system will continue allocating 512-byte blocks to it until it acquires a full 4KB of data
  - At that point, FFS will find a 4KB block, copy the sub-blocks into it, and free the sub-blocks for future use.

How to address internal fragmentation?

- Sub-blocks within a block
  - 512-byte little blocks
- How does it work?
  - If you created a small file (say 1KB in size), it would occupy two sub-blocks and thus not waste an entire 4KB block
  - As the file grew, the file system will continue allocating 512-byte blocks to it until it acquires a full 4KB of data
  - At that point, FFS will find a 4KB block, copy the sub-blocks into it, and free the sub-blocks for future use.
- Problem: lots of extra work (esp. extra I/O for the copy)
  - FFS generally avoids sub-blocks for performance

FFS Results

Table IIa. Reading Rates of the Old and New UNIX File Systems

	Type of file system	Processor and bus measured	Speed (Kbytes/s)	Read bandwidth %	% CPU
UFS	Old 1024	750/UNIBUS	29	29/983 3	11
	New 4096/1024	750/UNIBUS	221	221/983 22	43
FFS	New 8192/1024	750/UNIBUS	233	233/983 24	29
	New 4096/1024	750/MASSBUS	466	466/983 47	73
	New 8192/1024	750/MASSBUS	466	466/983 47	54

Table IIb. Writing Rates of the Old and New UNIX File Systems

	Type of file system	Processor and bus measured	Speed (Kbytes/s)	Write bandwidth %	% CPU
UFS	Old 1024	750/UNIBUS	48	48/983 5	29
	New 4096/1024	750/UNIBUS	142	142/983 14	43
FFS	New 8192/1024	750/UNIBUS	215	215/983 22	46
	New 4096/1024	750/MASSBUS	323	323/983 33	94
	New 8192/1024	750/MASSBUS	466	466/983 47	95

FFS Results

Table IIa. Reading Rates of the Old and New UNIX File Systems

	Type of file system	Processor and bus measured	Speed (Kbytes/s)	Read bandwidth %	% CPU
UFS	Old 1024	750/UNIBUS	29	29/983 3	11
	New 4096/1024	750/UNIBUS	221	221/983 22	43
FFS	New 8192/1024	750/UNIBUS	233	233/983 24	29
	New 4096/1024	750/MASSBUS	466	466/983 47	73
	New 8192/1024	750/MASSBUS	466	466/983 47	54

writes in FFS are slower than reads

Table IIb. Writing Rates of the Old and New UNIX File Systems

	Type of file system	Processor and bus measured	Speed (Kbytes/s)	Write bandwidth %	% CPU
UFS	Old 1024	750/UNIBUS	48	48/983 5	29
	New 4096/1024	750/UNIBUS	142	142/983 14	43
FFS	New 8192/1024	750/UNIBUS	215	215/983 22	46
	New 4096/1024	750/MASSBUS	323	323/983 33	94
	New 8192/1024	750/MASSBUS	466	466/983 47	95

FFS Results

Table IIa. Reading Rates of the Old and New UNIX File Systems

	Type of file system	Processor and bus measured	Speed (Kbytes/s)	Read bandwidth %	% CPU
UFS	Old 1024	750/UNIBUS	29	29/983 3	11
	New 4096/1024	750/UNIBUS	221	221/983 22	43
FFS	New 8192/1024	750/UNIBUS	233	233/983 24	29
	New 4096/1024	750/MASSBUS	466	466/983 47	73
	New 8192/1024	750/MASSBUS	466	466/983 47	54

not the case for old FS

writes in FFS are slower than reads

Table IIb. Writing Rates of the Old and New UNIX File Systems

	Type of file system	Processor and bus measured	Speed (Kbytes/s)	Write bandwidth %	% CPU
UFS	Old 1024	750/UNIBUS	48	48/983 5	29
	New 4096/1024	750/UNIBUS	142	142/983 14	43
FFS	New 8192/1024	750/UNIBUS	215	215/983 22	46
	New 4096/1024	750/MASSBUS	323	323/983 33	94
	New 8192/1024	750/MASSBUS	466	466/983 47	95

FFS Results

Table IIa. Reading Rates of the Old and New UNIX File Systems

	Type of file system	Processor and bus measured	Speed (Kbytes/s)	Read bandwidth %	% CPU
UFS	Old 1024	750/UNIBUS	29	29/983 3	11
	New 4096/1024	750/UNIBUS	221	221/983 22	43
FFS	New 8192/1024	750/UNIBUS	233	233/983 24	29
	New 4096/1024	750/MASSBUS	466	466/983 47	73
	New 8192/1024	750/MASSBUS	466	466/983 47	54

not the case for old FS

writes in FFS are slower than reads

Table IIb. Writing Rates of the Old and New UNIX File Systems

	Type of file system	Processor and bus measured	Speed (Kbytes/s)	Write bandwidth %	% CPU
UFS	Old 1024	750/UNIBUS	48	48/983 5	29
	New 4096/1024	750/UNIBUS	142	142/983 14	43
FFS	New 8192/1024	750/UNIBUS	215	215/983 22	46
	New 4096/1024	750/MASSBUS	323	323/983 33	94
	New 8192/1024	750/MASSBUS	466	466/983 47	95

CPU load is fine given that UFS is way too slow!

FFS Results

- 20-40% of disk bandwidth for large reads/writes
- 10-20x original UNIX speeds
- 10% of total disk space unusable (except at 50% performance price)
- SOTA file systems that heavily leverage FFS
  - UFS used by FreeBSD, OpenBSD, NetBSD..
  - ZFS developed by Sun Microsystems and maintained by OpenZFS

FFS Results

- 20-40% of disk bandwidth for large reads/writes
- 10-20x original UNIX speeds
- 10% of total disk space unusable (except at 50% performance price)

Lesson: don't ignore underlying hardware characteristics

- SOTA file systems that heavily leverage FFS
  - UFS used by FreeBSD, OpenBSD, NetBSD..
  - ZFS developed by Sun Microsystems and maintained by OpenZFS

File operations still expensive

- How many I/O operations (seeks) to create a new file?

File operations still expensive

- How many I/O operations (seeks) to create a new file?
  - New file, needs a new inode
  - But at least a block of data too
  - Check and update the inode and data bitmap (eventually have to be written to disk)
  - Not done yet – need to add it to the directory (and update the directory inode and the directory data block – may need to split if its full)…
  - Whew!! How does all of this even work?
- Problems of FFS?

File operations still expensive

- How many I/O operations (seeks) to create a new file?
  - New file, needs a new inode
  - But at least a block of data too
  - Check and update the inode and data bitmap (eventually have to be written to disk)
  - Not done yet – need to add it to the directory (and update the directory inode and the directory data block – may need to split if its full)…
  - Whew!! How does all of this even work?
- Problems of FFS?
  - Lots of little writes! (many seeks for writes to random locations)

Discussion Questions

Why did FFS introduce *cylinder groups* as a core design optimization?

A. To distribute data blocks evenly across all disk platters
B. To eliminate the need for inodes in the file system
C. To cluster related files and directories together to reduce seek time and improve spatial locality
D. To enforce stricter file permissions at the hardware level

Discussion Questions

FFS employs both *global* and *local* block allocation policies. What is the correct role of the *local* policy?

A. It chooses which directory the file should belong to
B. It determines the backup superblock position
C. It selects the best block within a cylinder group for allocation
D. It handles segment cleaning across cylinder groups

Log-Structured File System (LFS)

The Design and Implementation of a Log-Structured File System

Mendel Rosenblum and John K. Ousterhout

Electrical Engineering and Computer Sciences, Computer Science Division
University of California
Berkeley, CA 94720
mendel@sprite.berkeley.edu, ouster@sprite.berkeley.edu

(some slides credit to Christo Wilson)

Motivation

- How has computer hardware been evolving?
  - RAM has become cheaper and grown larger :)
  - Random access seek times have remained very slow :(
- This changing dynamic alters how disks are used
  - More data can be cached in RAM = less disk reads
  - Thus, writes will dominate disk I/O
- Can we create a file system that is optimized for sequential writes?

Log-Structured File System (LFS)

- Key idea: buffer all writes (including meta-data) in memory (log)
  - Write these long segments to disk sequentially
  - Treat the disk as a circular buffer, i.e. don't overwrite
- Characteristics:
  - All writes are large and sequential
  - Not necessarily good for reads, but trends help
    - Rely on a large memory to provide fast access through caching
  - Data layout on disk has "temporal locality" (good for writing), rather than "logical locality" (good for reading)
    - Why is this a better? Because caching helps reads but not writes!

Treating the Disk as a Log

- Data and meta-data get appended to a log
- Stale data isn't overwritten, its replaced



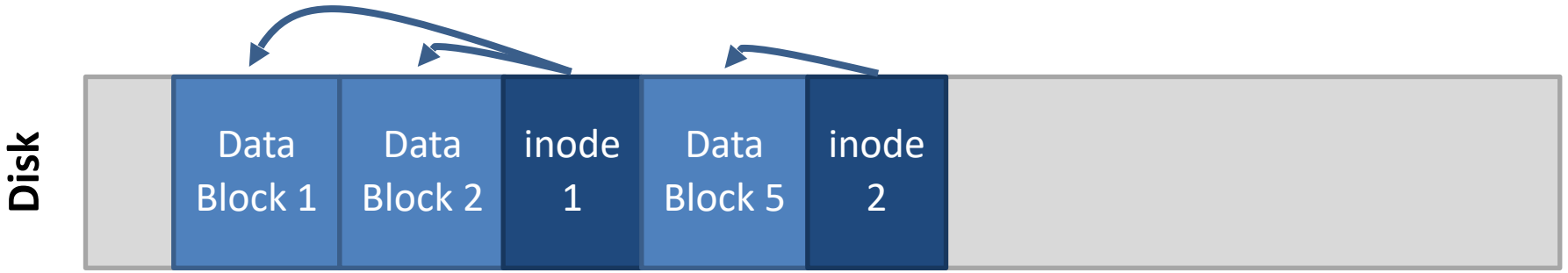
Treating the Disk as a Log

- Data and meta-data get appended to a log
- Stale data isn't overwritten, its replaced



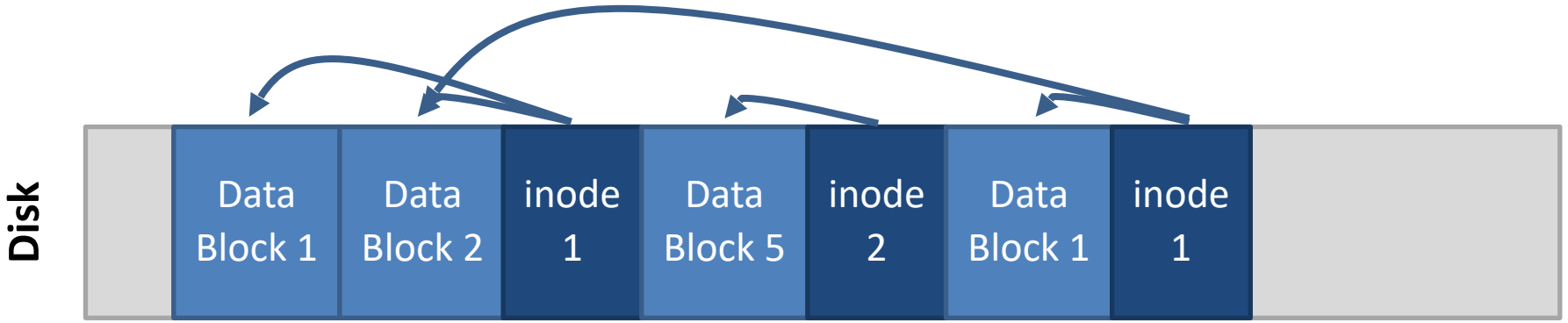
Treating the Disk as a Log

- Data and meta-data get appended to a log
- Stale data isn't overwritten, its replaced



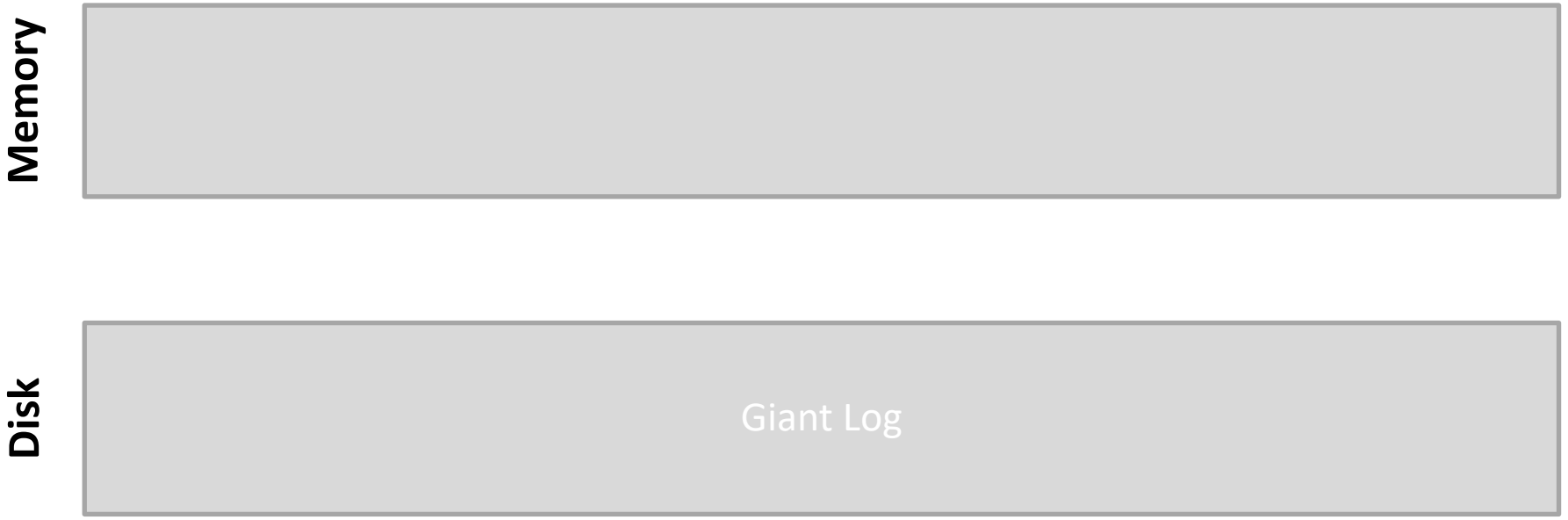
Treating the Disk as a Log

- Data and meta-data get appended to a log
- Stale data isn't overwritten, its replaced



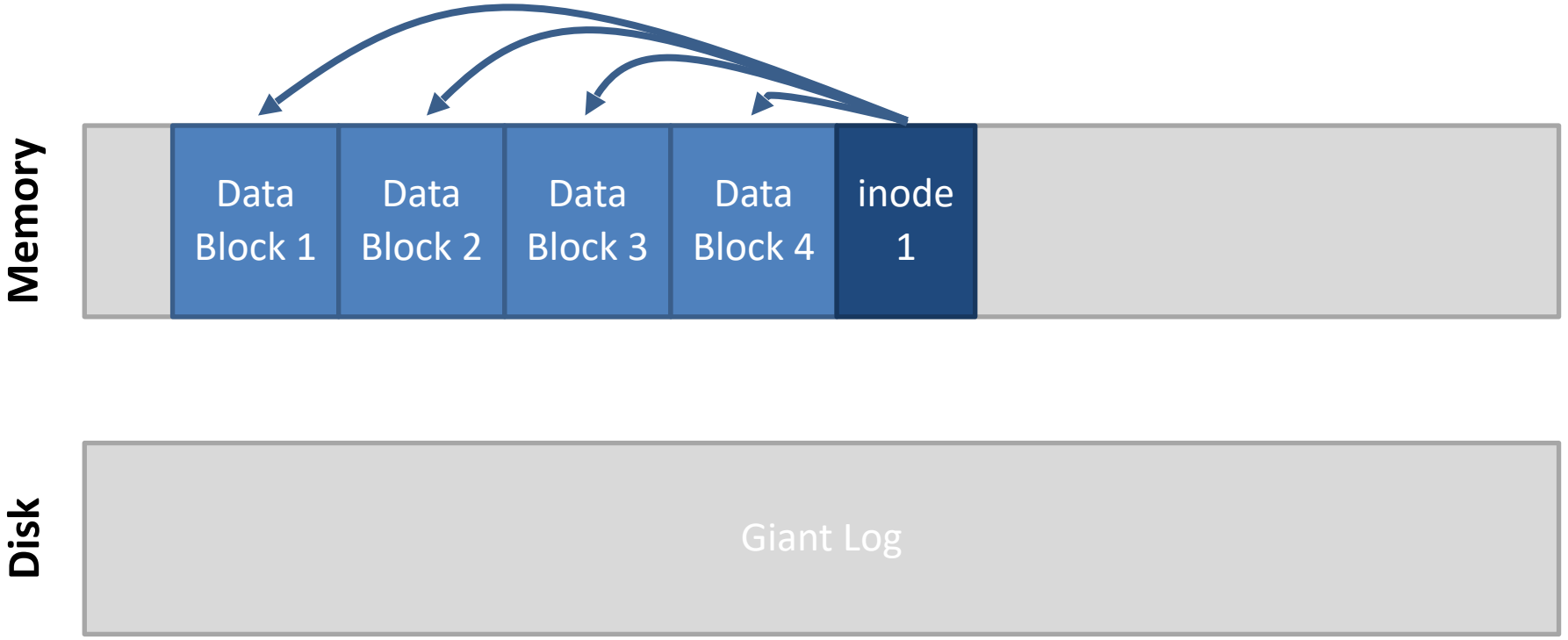
Buffering Writes

- LFS buffers writes in-memory into chunks



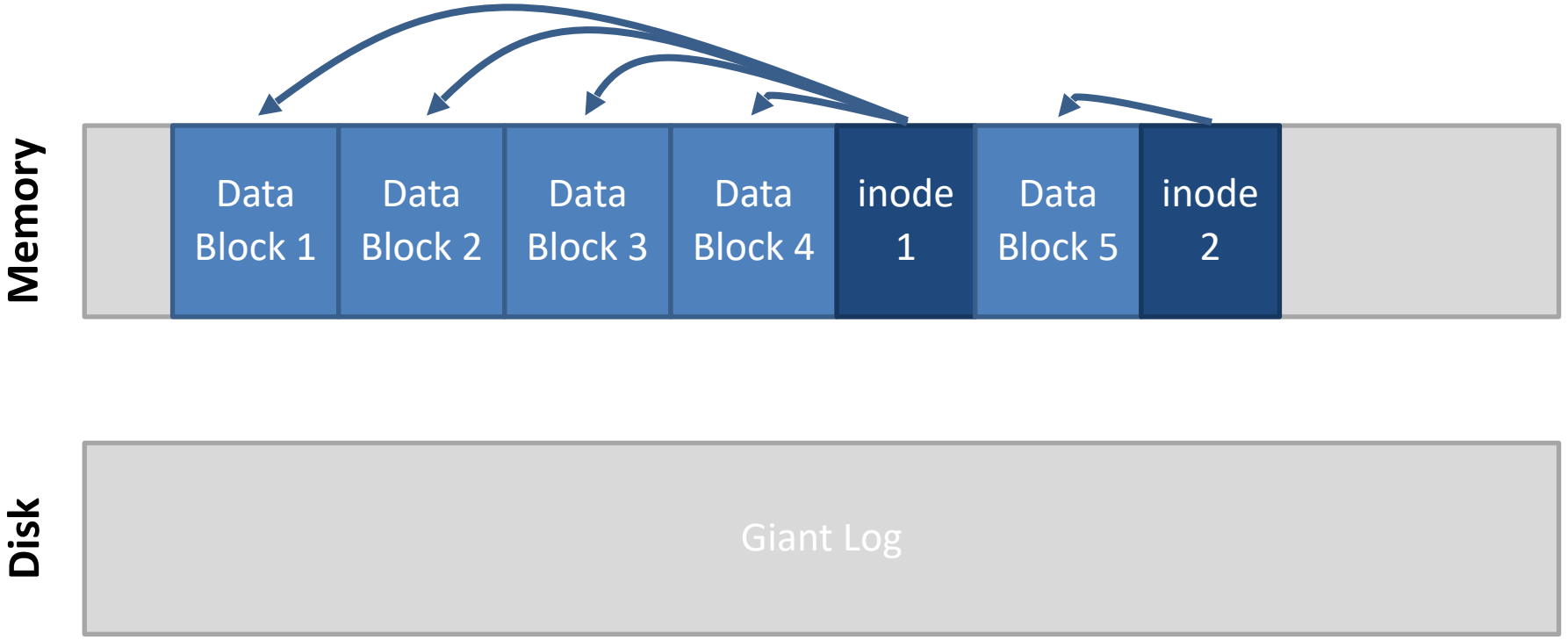
Buffering Writes

- LFS buffers writes in-memory into chunks



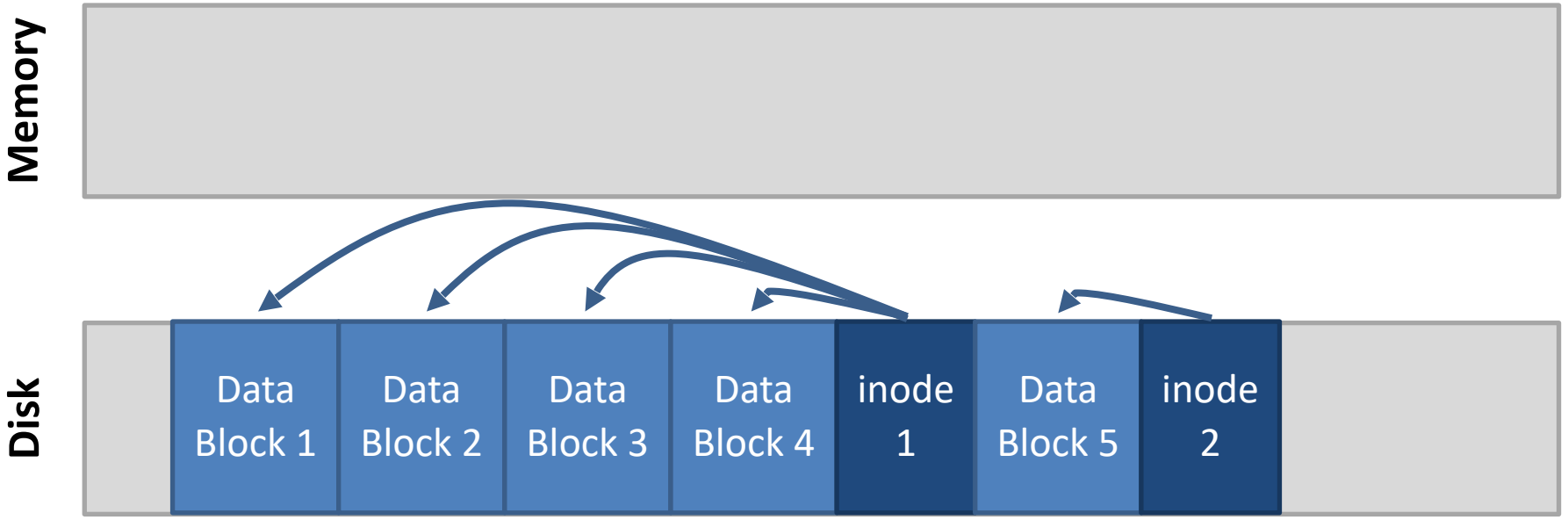
Buffering Writes

- LFS buffers writes in-memory into chunks



Buffering Writes

- LFS buffers writes in-memory into chunks



- Chunks get appended to the log once they are sufficiently large

How to Find a File?

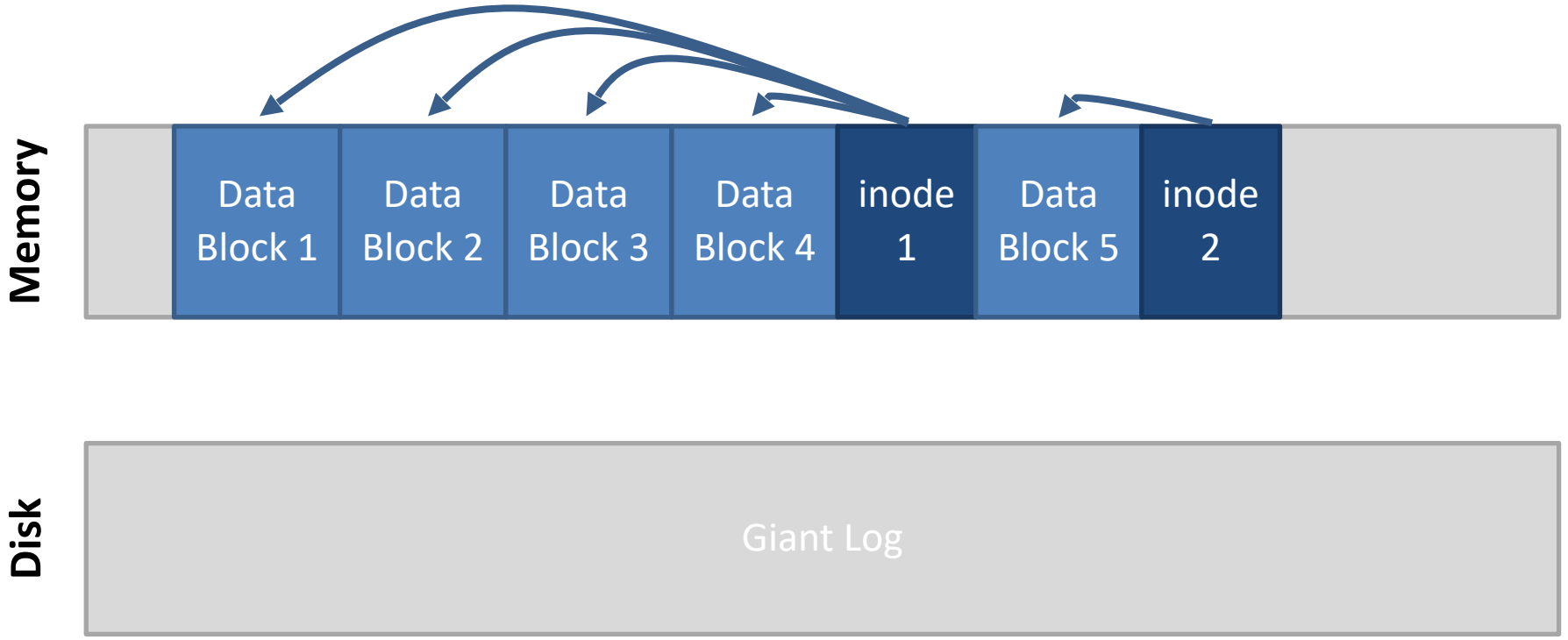
- In a typical file system, the inodes are stored at fixed locations (relatively easy to find)
- How do you find inodes in the log?
  - Remember, there may be multiple copies of a given inode (stale data isn't overwritten)

How to Find a File?

- In a typical file system, the inodes are stored at fixed locations (relatively easy to find)
- How do you find inodes in the log?
  - Remember, there may be multiple copies of a given inode (stale data isn't overwritten)

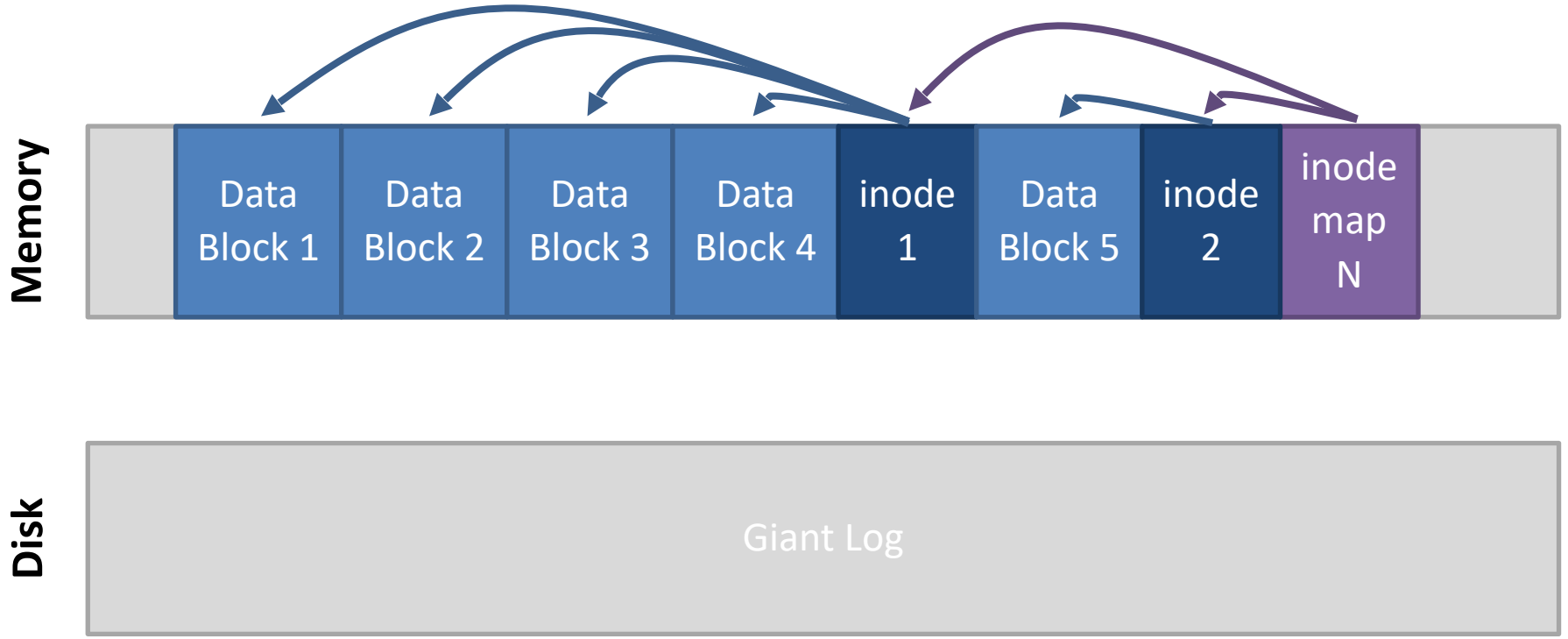
- Solution: inode map
  - inode map (imap): keeps track of the location of inodes
  - Anytime an inode is written to disk, the imap is updated
  - Most of the time the imap is in memory, so access is fast
  - Updated imap is saved as part of the log!

inode Maps



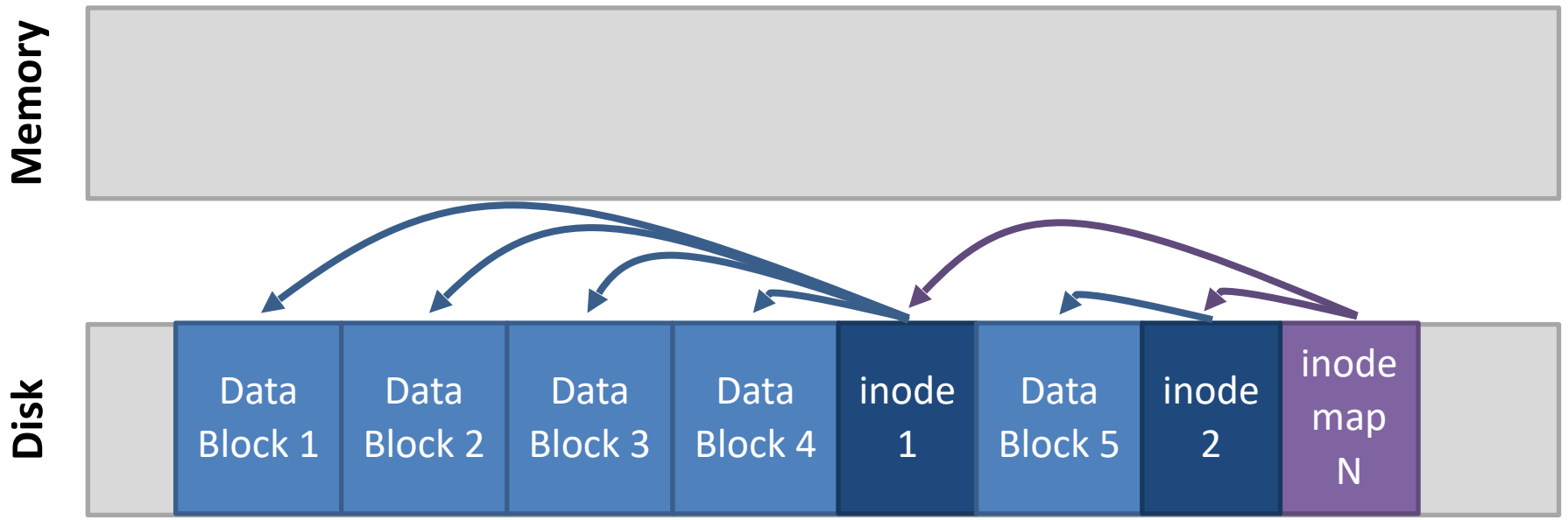
- New problem: imap is scattered throughout the log
  - How do we find the most up-to-date pieces?

inode Maps



- New problem: imap is scattered throughout the log
  - How do we find the most up-to-date pieces?

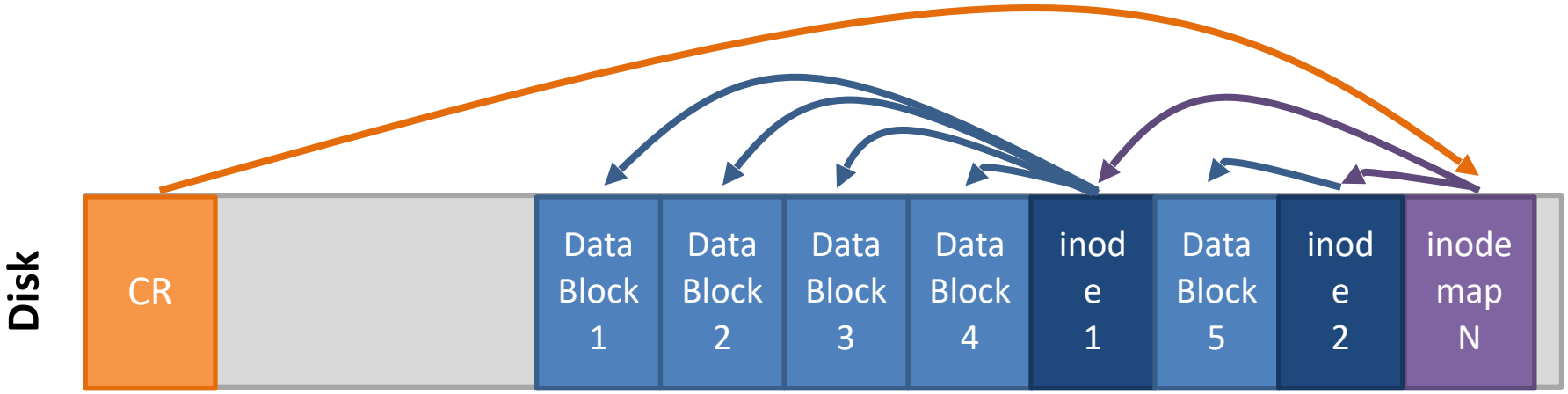
inode Maps



- New problem: imap is scattered throughout the log
  - How do we find the most up-to-date pieces?

Checkpoint Region

- Checkpoint Region (CR): points to the locations of the up-to-date inode maps
  - CR is part of the superblock but cached in memory
    - Written periodically to disk, say ~30 seconds
  - Also serves as an indicator of a stable point in the file system for crash recovery
  - Only part of LFS that isn't maintained in the log

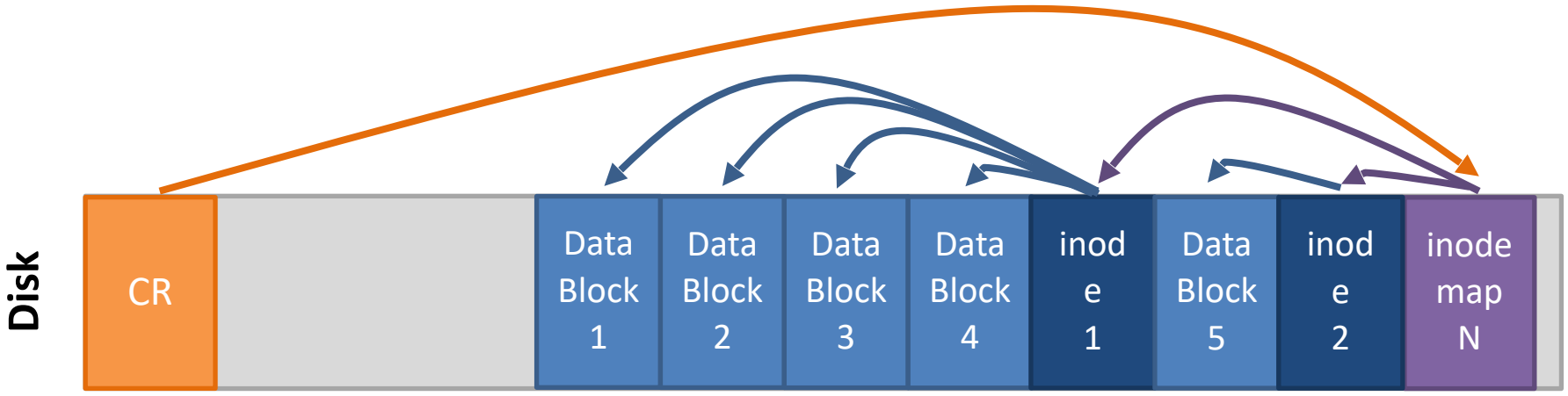


How to Read a File in LFS

- Read the CR, use it to read and cache the imap
- After that, it is identical to FFS
- Are reads fast?

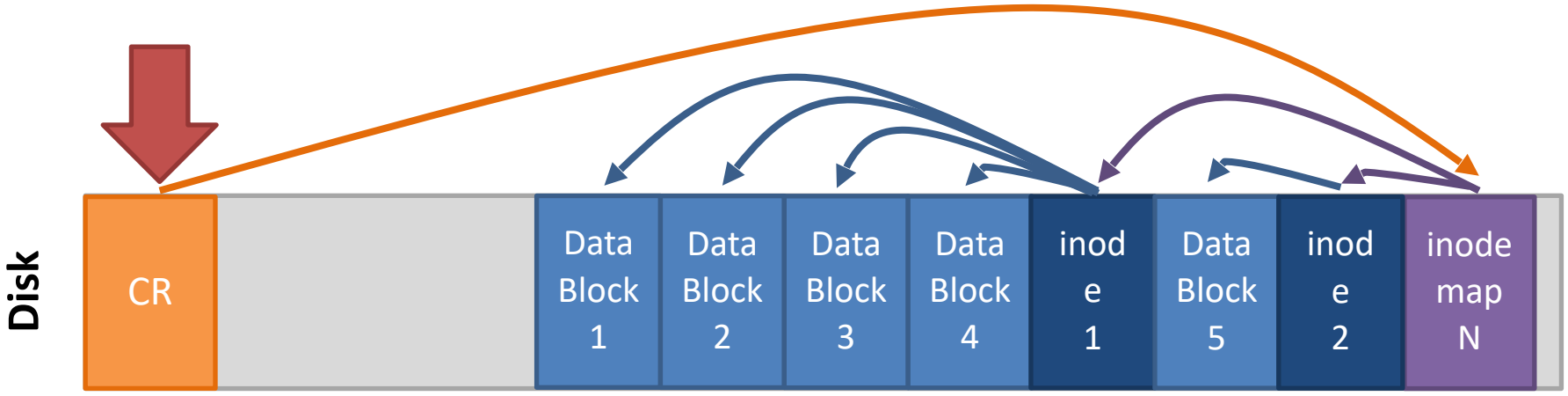
Example

- Suppose you want to read inode 1



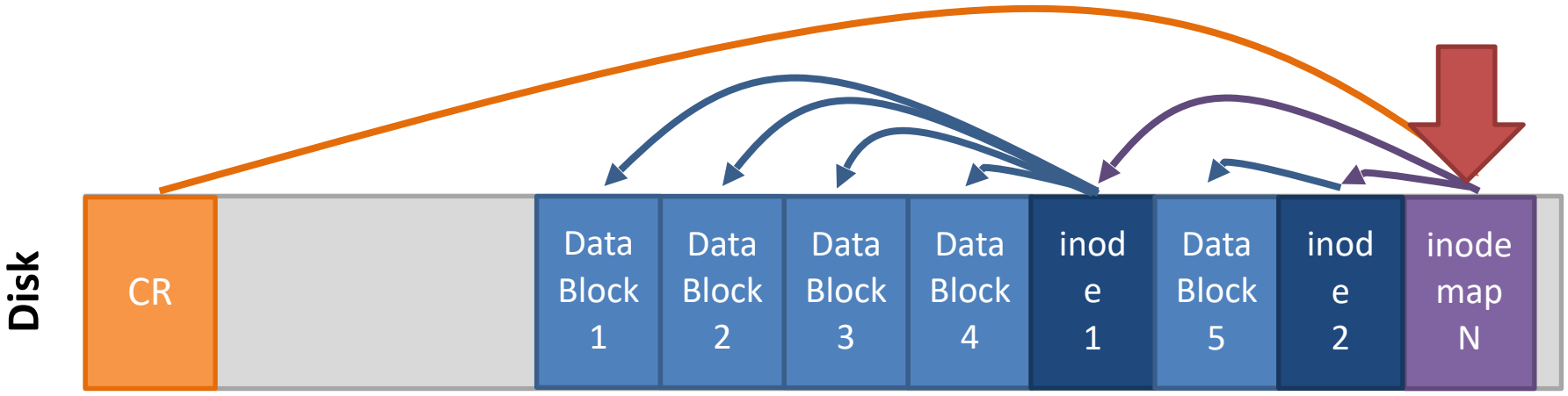
Example

- Suppose you want to read inode 1
  1. Look up inode 1 in the checkpoint region
     - inode map containing inode 1 is in sector *X*



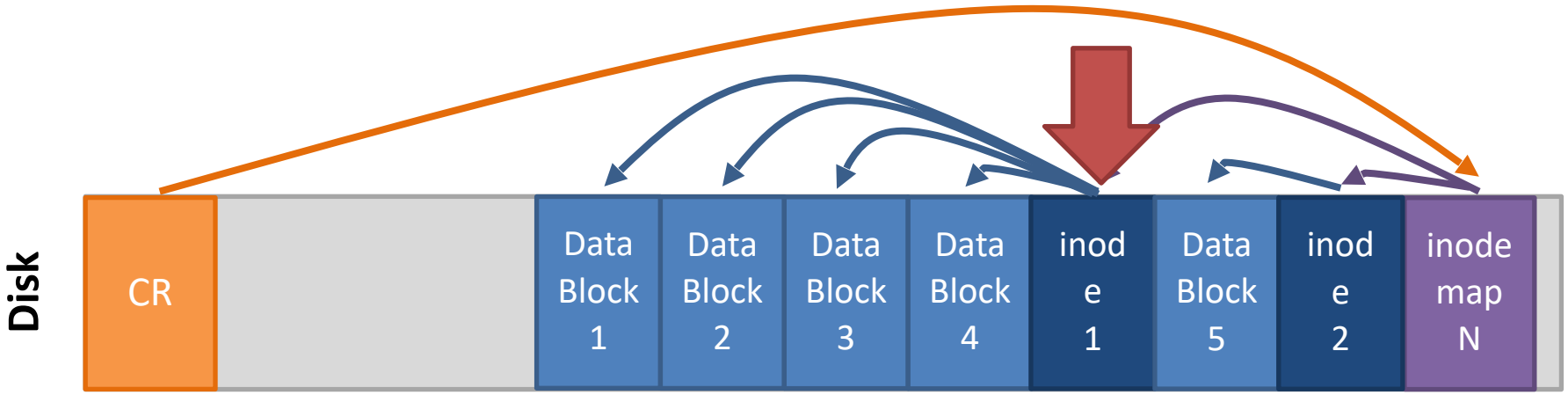
Example

- Suppose you want to read inode 1
  1. Look up inode 1 in the checkpoint region
     - inode map containing inode 1 is in sector *X*
  2. Read the inode map at sector *X*
     - inode 1 is in sector *Y*



Example

- Suppose you want to read inode 1
  1. Look up inode 1 in the checkpoint region
     - inode map containing inode 1 is in sector *X*
  2. Read the inode map at sector *X*
     - inode 1 is in sector *Y*
  3. Read inode 1
     - File data is in sectors *A*, *B*, *C*, etc.



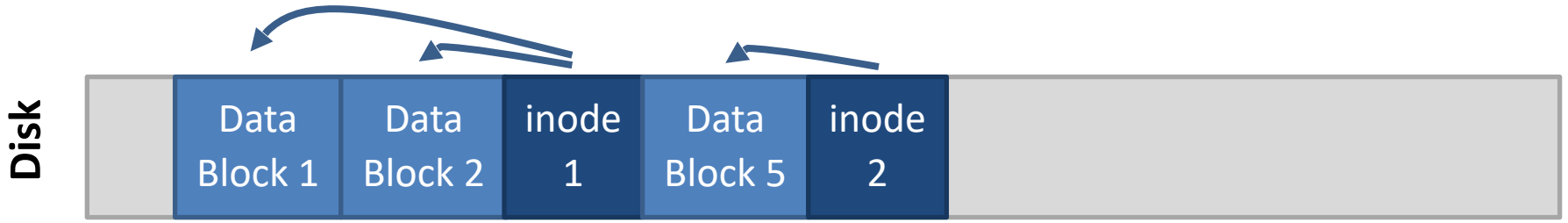
Discussion Questions

In LFS, what is the *checkpoint region* used for?

A. To store immutable copies of all past inode maps for snapshotting
B. To cache read-heavy segments in RAM for faster access
C. To track the location of the latest inode map blocks for crash recovery and log traversal
D. To identify data blocks marked for read-ahead during sequential reads

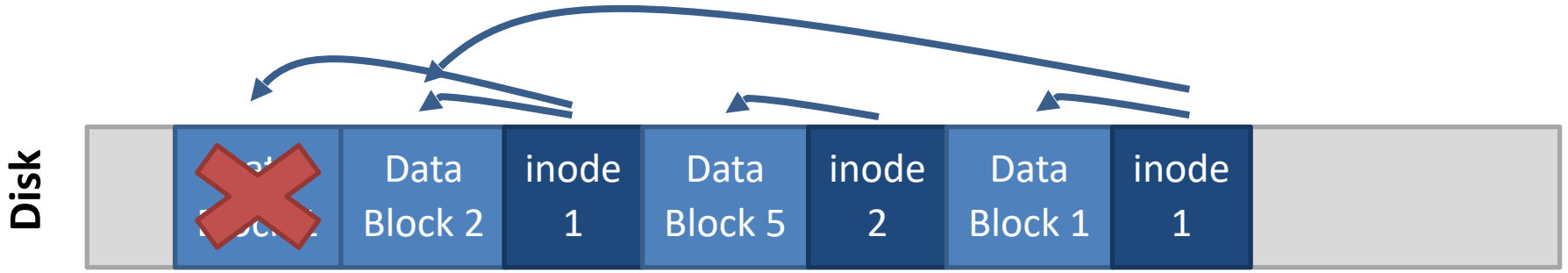
Segment cleaning/Garbage collection

- Over time, the log is going to fill up with stale data
  - Highly fragmented: live data mixed with stale data
- Reclaim invalidated segments in the log once the latest updates are checkpointed



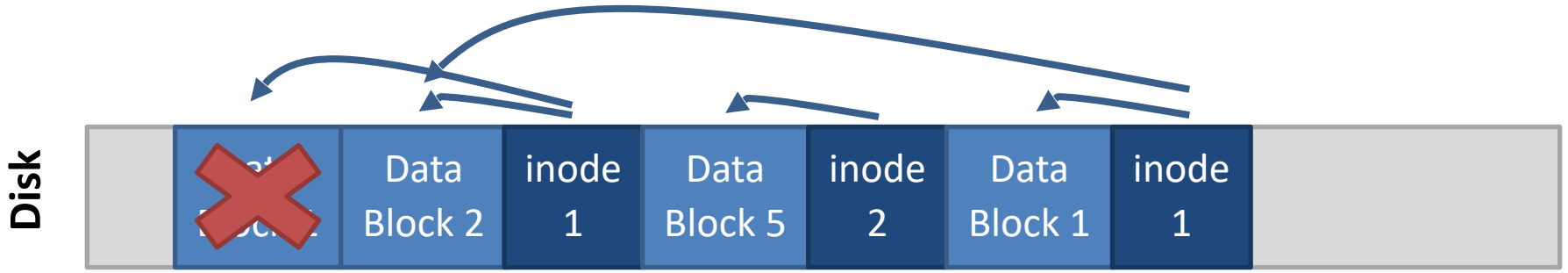
Segment cleaning/Garbage collection

- Over time, the log is going to fill up with stale data
  - Highly fragmented: live data mixed with stale data
- Reclaim invalidated segments in the log once the latest updates are checkpointed



Segment cleaning/Garbage collection

- Over time, the log is going to fill up with stale data
  - Highly fragmented: live data mixed with stale data
- Reclaim invalidated segments in the log once the latest updates are checkpointed



- Rearrange the data allocation to make continuous segments
- Must reserve enough space on the disk
  - Otherwise, every writes will trigger garbage collection
  - Sink the write performance

LFS Results

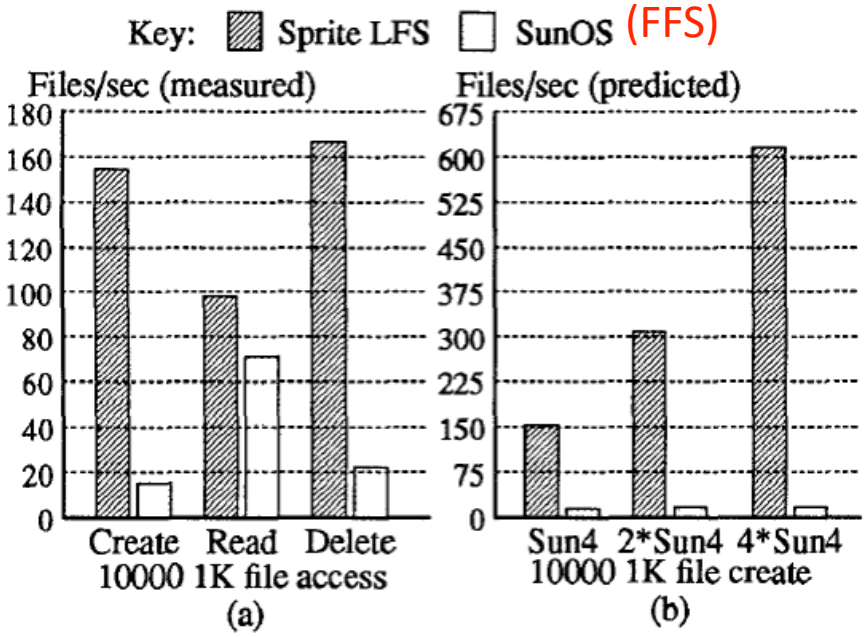


Figure 8 — Small-file performance under Sprite LFS and SunOS.
Figure (a) measures a benchmark that created 10000 one-kilobyte files, then read them back in the same order as created, then deleted them. Speed is measured by the number of files per second for each operation on the two file systems. The logging approach in Sprite LFS provides an order-of-magnitude speedup for creation and deletion. Figure (b) estimates the performance of each system for creating files on faster computers with the same disk. In SunOS the disk was 85% saturated in (a), so faster processors will not improve performance much. In Sprite LFS the disk was only 17% saturated in (a) while the CPU was 100% utilized; as a consequence I/O performance will scale with CPU speed.

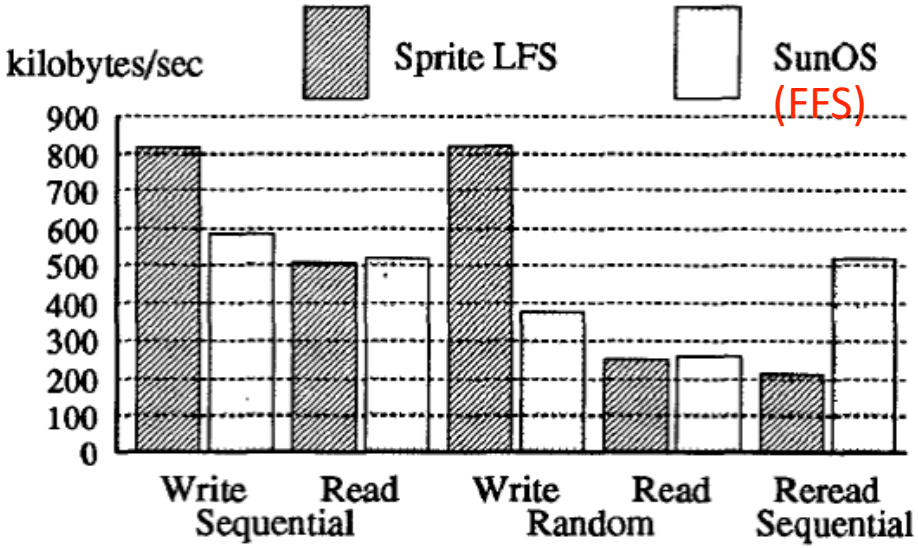


Figure 9 — Large-file performance under Sprite LFS and SunOS.
The figure shows the speed of a benchmark that creates a 100-Mbyte file with sequential writes, then reads the file back sequentially, then writes 100 Mbytes randomly to the existing file, then reads 100 Mbytes randomly from the file, and finally reads the file sequentially again. The bandwidth of each of the five phases is shown separately. Sprite LFS has a higher write bandwidth and the same read bandwidth as SunOS with the exception of sequential reading of a file that was written randomly.

An Idea Whose Time Has Come

- LFS seems like a very strange design
  - Totally unlike traditional file system structures
  - Doesn't map well to our ideas about directory hierarchies
- Initially, people did not like LFS
- However, today it's features are widely used

File Systems for SSDs

- SSD hardware constraints
  - To implement wear leveling, writes must be spread across the blocks of flash
  - Periodically, old blocks need to be garbage collected to prevent write-amplification
- Does this sounds familiar?
- LFS is the ideal file system for SSDs!
- Internally, SSDs manage all files in a LFS-like manner
  - This is transparent to the OS and end-users
  - Ideal for wear-leveling and avoiding write-amplification

Copy-on-write

- Modern file systems incorporate ideas from LFS
- Copy-on-write semantics
  - Updated data is written to empty space on disk, rather than overwriting the original data
  - Helps prevent data corruption, improves sequential write performance
- Pioneered by LFS, now used in ZFS and btrfs
  - btrfs for the next default file system in Linux..?

Versioning File Systems

- LFS keeps old copies of data by default
- Old versions of files may be useful!
  - Example: accidental file deletion
  - Example: accidentally doing *open(file, 'w')* on a file full of data
- Turn LFS flaw into a virtue
- Many modern file systems are versioned
  - Old copies of data are exposed to the user
  - The user may roll-back a file to recover old versions